Lean Formalization Seminar

Lean is a programming language with an library called *Mathlib* that is used to express mathematical theorems in a format that can be verified programmatically. The Mathlib library provides the semantic structure and definitions for proofs and statements, while the Lean language verifies that the logic is consistent. This is possible thanks to Lean's "functional" paradigm (in the sense of f(x)), which allows libraries to act as an additional layer that redefines notation to a high degree.

1. Installation and Project Setup

You may install Lean to code on VScode by following the official guide.

1. **Create an empty project**: in VScode command `Lean 4: Project: Create Project Using Mathlib`. Depending on internet speed, this may take some time, but you can see a log of progress in real time.
2. **Open the folder**: use VScode `File: Open Folder`. Explore the files to get a sense of project structure, but only edit the ones in the `My_project_name/` folder, as the others are managed automatically.
3. **Import Mathlib**: you can import the mathematical library by writing `import Mathlib` as the first line in the `Basic.lean` file.
4. **Lean4 InfoView**: when opening `.lean` files, you can use `Lean 4: InfoView: Toggle InfoView` to understand the intermediate state of a proof. Placing the cursor in the middle of the proof will show the available information and the necessary steps next. If strange errors appear but the proof is correct, you can press the `Restart File` button

Formalization projects start with a blueprint, which gets filled progressively: a sketch of the idea can be found on Terence Tao's blog, while you may find here a list of exemplars and the installation guide. For reference, this is an overview of the available theorems,

2. Type System

LEAN's language is based on the axioms of `Dependent Type Theory`. At a very high level, this allows for the existence of functions where the value of the first parameter can affect the type, and thus the value, of the later parameters that a function takes in. In this context, there is no difference between a function in the mathematical sense and in the programming sense.

To help visualize this change, consider the difference between the standard Cartesian Product and the type-dependent product:

- *Cartesian Product*: $\{(a, b) : a \in A, \quad b \in B\}$, here we combine two sets in a completely independent way.
- *Dependent Product*: $\{(a, b) : a \in A, \quad b \in B_a\}$, in this case the value of a directly affects the type, and thus possible values of b. Note that this assumes the existence of a function of type $A \to B$, where B is a set of possible types.

This allows for functions to behave differently based on their inputs, but also to construct high level arguments by abstracting from the details. Just like in mathematics, where the same proof can be applied to all objects that share a specific algebraic structure, we can imagine a proof like a function that first takes as an argument the type of the object, followed by some fixed assumptions on the objects, which depend on the object, but may be imposed for all objects in a more general collection.

The second peculiarity of Mathlib's system is the existence of an infinite family of `Universes`. In Mathlib, all objects that one normally interacts with are of `Type 0`, while `Type 0` is itself of type `Type 1`. This process is true inductively, and it has the purpose of avoiding inconsistencies such as `Russell's Paradox`: "Does the set of all sets that do not contain itself, contain itself?". In this case, the set does and does not contain itself. with the hierarchy of types, however, the collection of all sets of a given type lives in a higher universe, so a type cannot contain itself and the paradox is avoided. In fact, an object that refers to a Type, gets automatically elevated to the new type.

For instance, consider the notion of a linear function. Since linearity can be defined for a wide class of objects, the same proof may be used on different spaces, by simply fixing the overarching space, and then defining a function on this space. This would be an example of type dependence.

For Mathlib formalizations, we must consider two fundamental objects:

- **Proposition**: object that expresses a statement. They are constructed by combining basic facts through logic. To help visualize this, consider the statement `n=2 => n even`, which is built by mapping statements `n=2` and `n even` through an implication function taking the form `imply : Statement x Statement -> Statement`. Note that false statements may also be constructed in this way, thus we need another object to verify a statement.
- **Proof**: object that represents a series of steps that results in the construction of a Proposition. In Mathlib, a proof is an object of type proposition, and it is at the same time a function that "transforms" some assumptions into a series of conclusions. Following the previous example, a proof that `n=2` implies `n even` would be an object of type "Statement that `n=2 => n even`", but it also may be equivalently formulated as a function that given an object of type `n=2` (that is a proof of the statement), constructs an object of type `n even`.

Proof Irrelevance: when verifying the correctness of a statement, the Lean language verifies that each intermediate step follows from the previous one, by simply checking that the types of the proofs are always compatible. In this fashion, all proofs of a statement are equivalent for the sake of verifying the statement.

Following this formalism, **axioms** are simply a proof for a statement, where we do not give details for the intermediate steps, but accept them as a given. For example, an axiom may take the role of a proof in the intermediate steps of a demonstration.

3. The Check Command

We can use `#check obj` to verify or infer the type of a variable. This is useful to better understand the language, identify the meaning of a term, or to better understand what term is being introduced in a proof. In a similar fashion it is also possible to expand the definition of a term with `ctrl/cmd + click`.

```
variable (a b c : ℝ)

#check a
#check a + b
#check (a : ℝ)
#check mul_comm a b
#check mul_comm
#check (Type 20 → Type 21)
```

4. Functional Syntax

The syntax of Mathlib is simple, but all-encompassing by using **syntax overloading**: unicode characters like $(\forall, \exists, \mathbb{N}, \mathbb{Z}, \mathbb{Q}, \mathbb{R}, \leq, \neq, \in, \sum, \prod, \int \ldots)$ are used to represent complex concepts concisely. Most symbols can be inserted by simply writing "\" and the character

shorthand, but more can be defined as needed. Moreover, hovering the mouse over a symbol will show the proper shorthand.

The fundamental unit of knowledge in Mathlib is the `value : type` pair, which is used to construct all other complex functions. Consider the following definition of a "function":

```
theorem Complex.cos_add_sin_mul_I_pow (n : ℕ) (z : ℂ) :
  (cos z + sin z * I) ^ n = cos (↑n * z) + sin (↑n * z) * I
```

Here, the **theorem** keyword explains the definition, the name is `Complex.cos_add_sin_mul_I_pow`, and the type of the arguments is one variable `n` of type natural number and one variable `z` of type complex number, while the type of the output of the function is the statement of *De Moivre's theorem*. Observe also the presence of the `↑` character, which communicates upcasting: we convert the natural number n into a complex number, so that multiplication with a complex number is defined.

You may also encounter curly bracket arguments {...}, which fix the default values for the arguments, allowing them to be omitted, and square bracket arguments [...], which tell Lean to construct a type with some additional properties on the spot, typically for equipping parameters with some additional properties.

For example, the following is the definition of `Measurable`:

```
def Measurable {α : Type u_1} {β : Type u_2}
    [MeasurableSpace α] [MeasurableSpace β] (f : α → β) : Prop
```

To avoid confusion for cases where the same name is used in different theories, `namespaces` are used, which consist of areas of code with a fixed set of names.

5. Tactics

In order to lighten the burden of proving something one step at a time, Mathlib uses **meta-programming** extensively: there are algorithms that can quickly prove simple statements by taking the current state of the proof and attempting to construct a goal, which also gets indicated. This is very common with algebraic manipulations like equalities and inequalities, for which powerful tactics exist for rearranging terms within an expression. Tactics are introduced through the `by` keyword in the following way.

Some fundamental tactics, picked from the longer cheatsheet:

- `intro`: If the goal is `∀ x, P x` or `A → B`, introduces a new variable/hypothesis into the context (`intro x`, `intro h`), leaving the body as the new goal; supports patterns like `intro ⟨a, b⟩`.
- `unfold`: Expands a definition by replacing it with its body (`unfold f`, `unfold f g`); can target locations (`unfold f at h`).
- `rw`: Rewrites occurrences using equalities/iff (`rw [h]` left-to-right, `rw [← h]` right-to-left); can rewrite in hypotheses (`rw [h] at hx`) and chain rules (`rw [h1, h2]`).
- `ring`: Closes goals of the form `lhs = rhs` in commutative (semi)rings by normalizing both sides as polynomials in the ring operations; `ring!` is a more aggressive variant.
- `rel`: For relational goals like `≤` / `<`, transforms the goal using monotonicity/congruence lemmas and the inequalities you provide; e.g. `rel [h1, h2]` reduces the goal to simpler subgoals.
- `have`: Adds a new local fact to context (`have h : P := t` or `have h : P := by ...`); supports destructuring patterns like `have ⟨h1, h2⟩ := h`.

- `exact`: Closes the goal by providing a term whose type is definitionally equal to the goal (`exact t`).
- `simp`: Simplifies the goal (or hypotheses) using simp lemmas and definitional reductions; common forms: `simp`, `simp [h1, h2]`, `simp at h`, `simp [*]`, `simp only [lemmas...]`.
- `linarith`: Solves goals that are linear equalities/inequalities by combining linear (in)equalities from the context (derives contradictions and bounds automatically).

6. Basic Tactics

We first consider an overview of the basic tactics which can be used to prove standard arithmetic statements or algebraic manipulations

rewrite

We can use `rw` to substitute the hypothesis into an equation.

```
example (a b c : ℝ) : a * b * c = b * (a * c) := by
  rw [mul_comm a b]    -- first obtain that b * a = a * b from a * b
  rw [mul_assoc b a c] -- apply associativity as (a * b) * c = b * (a * c)

-- notably, we may also omit terms that are clear from context
example (a b c : ℝ) : a * b * c = b * c * a := by
  rw [mul_assoc] -- apply substitution, but with the inverted hypothesis
  rw [mul_comm] -- apply substitution, but with the inverted hypothesis
```

ring

The `ring` tactic helps to simplify a expression by performing simple operations like cancelling terms. It works by simply rearranging an expression to normal form.

```
example : (a + b) * (a - b) = a ^ 2 - b ^ 2 := by
  ring -- can be simplified directly

example (hyp : c = d * a + b) (hyp' : b = a * d) : c = 2 * a * d := by
  rw [hyp, hyp'] -- we simply apply both hypotheses
  ring           -- simplify using the ring tactic
```

relation

We can use the `rel` tactic to apply an inequality which we have as a hypothesis to a statement. As before, this is a fundamental building block.

```
example {r s : ℚ} (h1 : s + 3 ≥ r) (h2 : s + r ≤ 3) : r ≤ 3 :=
  calc
    r = (s + r + r - s) / 2 := by ring
    _ ≤ (3 + (s + 3) - s) / 2 := by rel [h1, h2]
    _ = 3 := by ring
```

Examples: First, Second, Third, Fourth

Use `norm_num` and `linarith [sq_nonneg (x - y)]` in exercises 3 and 4.

apply

The `apply` tactic can be used to invoke a theorem in a specific setting. It also allows some generality on exact use. Also, we can use `·` to guide the subgoals, helping steer the tactic in the right direction.

```
-- transitivity of the relation "smaller or equal to"
#check (le_trans : a ≤ b → b ≤ c → a ≤ c)

example (x y z : ℝ) (h₀ : x ≤ y) (h₁ : y ≤ z) : x ≤ z := by
  apply le_trans -- steer the tactic in two steps:
  · apply h₀
  · apply h₁
```

linear arithmetic

The `linarith` tactic can handle basic linear arithmetic, such as filling in the steps required to simplify an equality or a basic inequality.

```
example (h : 2 * a ≤ 3 * b) (h' : 1 ≤ a) (h'' : d = 2) : d + a ≤ 5 * b := by
  linarith -- simple enough that it can be solved directly
```

have

We can use the `have` tactice to introduce an intermediate goal, that is an additional hypothesis, in the middle of the proof.

```
example {a b : ℝ} : 2 * a * b ≤ a^2 + b^2 := by
  have h : 0 ≤ a^2 - 2 * a * b + b^2 := -- define the new hypothesis
    calc                                -- prove it with calc
      0 ≤ (a - b)^2 := by apply pow_two_nonneg
      _ = a^2 - 2 * a * b + b^2 := by ring

  linarith -- close the goal with linarith
```

More powerful tactics also exists, with the tradeoff that greater automation results in slower verification. For this reason, powerful tactics are often replaced with more precise solutions in the foundational theorems in Mathlib.

Examples: First, Second, Third, Fourth

Reverse Direction

This is not a tactic, but it is related to many application. Oftentimes, relations are applied in reverse order. This operation is carried out with the `←` symbol

```
example : (a + b) * (a + b) = a * a + 2 * (a * b) + b * b :=
  calc
    (a + b) * (a + b) = a * a + b * a + (a * b + b * b) := by
      rw [mul_add, add_mul, add_mul]
    _ = a * a + (b * a + a * b) + b * b := by
      rw [← add_assoc, add_assoc (a * a)] -- reverse associativity
    _ = a * a + 2 * (a * b) + b * b := by
      rw [mul_comm b a, ← two_mul] -- reverse order
```

Listing 1: A more complex example using a couple of theorems

7. Calc Mode

When we write `calc` after the `by` keyword, we are switching to a step-by-step derivation which allows us to concatenate expressions with justifications in a more readable way. Each line proves that the current expression is related to the previous one, and Lean combines the lines using transitivity. It is mainly used for equalities, inequalities, and other transitive relations

We should now have enough of an understanding to read a more complex proof, and understand it to some extent. The following example proves that the affine combination between Lotteries (a probability distribution over a countable support X) has probability that sums to 1.

```
private lemma mix_sum_one (p q : Lottery X) (α : Real) :
    -- sum over the support of X, where x is the index
    ∑ x, (α * p.val x + (1 - α) * q.val x) = 1 := by

  -- use the calc tactic to prove by direct simplification
  calc
    -- rewrite the left side, and reach the right side
    ∑ x, (α * p.val x + (1 - α) * q.val x)

    -- the new goal is written on the next line,
    -- but the justification comes from the tactic after `by`
    _ = α * ∑ x, p.val x + (1 - α) * ∑ x, q.val x := by

        -- the rewrite tactic, which uses `Finset` properties
        rw [Finset.sum_add_distrib, Finset.mul_sum, Finset.mul_sum]

    -- use rewrite again with the properties of `p` and `q` Lotteries
    _ = α * 1 + (1 - α) * 1 := by rw [p.property.2, q.property.2]

    -- use the `ring` tactic to do simple algebraic cancellations
    _ = 1 := by ring
```

Listing 2: von Neumann-Morgenstern Utility Theorem - lemma

7.1. Searching for Tactics

To help navigate Mathlib's multitude of theorems and tactics, there are many resources that can be used to identify their exact names:

- `apply?` : the InfoView panel can suggestion an appropriate tactic.

```
example : 0 ≤ a ^ 2 := by
  apply?
```

$\Longrightarrow$

```
Try this:
  [apply] exact sq_nonneg a
```

- Guessing: Lean4 naming conventions help maintain consistency across libraries, allowing users to guess exact theorem names.
- `Search Engines` : Lean Explore and Moogle provide semantic search capabilities.
- `Zulip` : the official Lean forum, provides users with a `Is there code for X?` page.

8. Additional References

Lean may be used online, with no installation at this link.

8.1. Sources

- *Mathematics in Lean*, leanprover-community, Online book
- *Lean 4 Tactic Cheatsheet*, Floris van Doorn, Source
- *The Mechanics of Proof*, Heather Macbeth. Online Book
- *Mechanized Proofs of the vNM Utility Theorem*, Jingyuan Li, Code, Paper
- *Github Lean Project Template*, leanprover-community, LeanProject Template

8.2. Complete Projects

- Analytic Number Theory Exponent Database by Terence Tao et al.
- Equational Theories by Terence Tao et al.
- Polynomial Freiman Ruzsa Conjecture by Terence Tao et al.
- Infinity Cosmos by Emily Riehl et al.
- ABC Exceptions by Bhavik Mehta et al.
- Proofs from THE BOOK by Moritz Firsching et al.
- Soundness of FRI by Bolton Bailey et al.
- Sphere Packing in 8 Dimensions by Maryna Viazovska et al.
- Groupoid Model of Homotopy Type Theory by Sina Hazratpour et al.
- Weil's Converse Theorem by Chris Birkbeck et al.
- Dirichlet Nonvanishing by Chris Birkbeck et al.
- Spectral Theorem by Oliver Butterley and Yoh Tanimoto.
- Automata Theory by Stefan Hetzl et al.
- Seymour's Decomposition Theorem by Ivan Sergeyev et al.
- NeuralNetworks by Matteo Cipollina.

8.3. Smaller Projects

- *Unit Fractions Density Conjecture*: git, paper, blueprint
- *Liquid Tensor*: git, paper and image, blueprint.
- *Arithmetic Progressions - Almost Periodicity*: home, documentation, git
- *Brauer Group and Galois Cohomology*: paper, code documentation, blueprint
- *Brownian Motion*: git, documentation, blueprint,
- *Toric Varieties*: home, paper without lean references, blueprint, upstream-ready
- *Combinatorial Games*: home, git,
- *Forbidden Matrix Theory*: overview, git, blueprint code.
- *∞-Cosmoi for Lean*: home, blueprint, docs

Related but through a different formalization language: Busy Beaver Challenge.